

Lab 06: R Functions

Parisa Charoensook (PID:A17939591)

Table of contents

Background	1
A first function	1
Q2. Write a generate_dna() function	3
Are Our peptides “unique in nature”?	7

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can be either “required” or “optional”. The later have fall-back default values that will be used if the user does not specifcy them.

```
add <- function(x,y=0) {  
  x+y}
```

Can we use our new function:

```
add (10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

Q1b: For your second version. adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10)). [1 pt]

```
add<- function(x, y=0, z=0, d=0) {  
  sum(x,y,z,d)  
}  
add(4,7)
```

```
[1] 11
```

```
add ( c(4,7,10))
```

```
[1] 21
```

```
add(1,2,3,-4)
```

```
[1] 2
```

Q1.C: Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return.

```
add<- function(x, y=0, z=0) {  
  sum(x,y,z)  
}
```

```
add <- function(x, y=0, z=0) {  
  sum(x,y,z)  
  cat("Is it break time yet?\n")  
}  
ans <- add(1,2,3)
```

```
Is it break time yet?
```

```
add(10,100)
```

Is it break time yet?

```
add <- function(x, y=0, z=0, d=0) {  
  sum(x,y,z,d)  
}  
add(4,7)
```

```
[1] 11
```

```
add(1,2,3,-4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the **return()** function call.

Q2. Write a `generate_dna()` function

A useful function here is the “base R” `sample()` function:

```
sample(1:5, size=60, replace=TRUE)
```

```
[1] 1 3 3 5 3 1 2 2 3 5 4 1 1 4 3 1 4 5 4 3 3 5 4 4 1 5 1 4 5 1 3 3 3 4 5 5 2 3  
[39] 3 5 4 2 3 1 1 3 5 1 3 2 1 5 3 1 4 5 3 5 3 3
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G” and “T”...

```
sample(x=c("A","C","G","T"), size = 10, replace=TRUE)
```

```
[1] "A" "C" "T" "G" "A" "G" "A" "C" "A" "A"
```

Q2.: Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
generate_dna <- function(len) {
  sample(x=c("A","C","G","T"), size = len, replace=TRUE)
}
```

```
generate_dna()
```

```
[1] "C" "C" "G" "G"
```

```
generate_dna(len=100)
```

```
[1] "T" "A" "C" "C" "G" "G" "A" "A" "T" "C" "T" "A" "T" "T" "T" "T" "C" "C"
[19] "A" "T" "C" "A" "A" "C" "G" "A" "C" "C" "A" "G" "A" "G" "C" "C" "C" "G"
[37] "T" "A" "T" "A" "T" "T" "A" "T" "T" "C" "C" "G" "C" "G" "C" "T" "G" "A"
[55] "A" "C" "A" "G" "A" "C" "C" "T" "C" "G" "T" "T" "A" "C" "A" "G" "T" "C"
[73] "A" "C" "G" "T" "C" "T" "A" "A" "T" "T" "C" "G" "G" "T" "A" "G" "A" "A"
[91] "G" "T" "A" "T" "T" "T" "G" "C" "G" "C"
```

Q2B: Your second version should *optionally* be able to return either a multi-element vector of singlecharacter nucleotides (as before) or a single character string (not a vector of single letters but a single vector of multiple letters). For example "AAGGCTG". [1 pt]

```
generate_dna <- function(len=10,single.element=TRUE) {

  ans <- sample(x=c("A","C","G","T"), size=len, replace=TRUE)
  #cat("Hello,,")

  if(single.element) {
    #cat("is it me you are looking for...")

    ans <- paste(ans, collapse = "")
  }
  return(ans)
}
```

```
paste( c("A","C","G"), collapse = "")
```

```
[1] "ACG"
```

Functions that could be useful here are `paste()`, `if()`, `cat()` and `return()`

```
generate_dna(single.element = TRUE)
```

```
[1] "CCCGAGTGAA"
```

Q2C: Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example:

```
>len9  
CGAAGGCTG
```

```
cat("hello \n there")
```

```
hello  
there
```

```
generate_dna <- function(len=10,single.element=TRUE) {  
  
  ans <- sample(x=c("A","C","G","T"), size=len, replace=TRUE)  
  
  if(single.element) {  
    #cat("is it me you are looking for...")  
    ans <- paste(ans, collapse = "")  
  }  
  ##Format as FASTA with an ID Line  
  cat( paste(">len",len,"\n", sep=""))  
  cat(ans)  
  cat("/n")  
}
```

```
x <- generate_dna(44)
```

```
>len44  
GTGCGGGTTGGTTTCCAGGATAACAGGTAAATTGTAAAAGTGGG/n
```

Q3. Write a function `generate__protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate__protein(6)` might return "WQRTAG". Your function should:

```
generate_protein <- function(len=9) {
  aa <- c("A","R","N","D","C","E","Q","G","H","I","L","K","M","F","P","S","T","W","Y","V")

  ans <- sample(x=aa, size=len, replace= TRUE)
  paste(ans,collapse= "")
}
```

```
generate_protein(5)
```

```
[1] "HDRTS"
```

Q4: Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the baseR function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

```
for (l in 6:13) {
  cat(">", l, "\n", sep="")
  cat( generate_protein(l), "\n" )
}
```

```
>6
IFNSQP
>7
PKCLEII
>8
DCYYQMPT
>9
HRPMDGDDM
>10
TMQKCRKHHF
>11
VHFRWNGDHSW
>12
IIFMCLSTYSHM
>13
HPRNKTKVNNCAL
```

```
generate_protein(5)
```

```
[1] "CIVQE"
```

Are Our peptides “unique in nature”?

Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

length	Ide	Cov	Unique
6	100	100	N
7	100	100	N
8	100	100	N
9	100	88	Y
10	100	90	Y
11	100	81	y
12	83	90	y
13	77	90	y

- My random peptides start looking unique in nature at about length 9. This is when it stops showing 100% coverage + 100% identity hit. This was found from lengths 6-8.
- Short random peptides are almost always found in nr because the total number of possible sequences is still relatively small compared with the enormous number of protein sequence in nature. But as peptide length increases, the sequence space grows exponentially as 20^L , so the number of possible sequences quickly becomes far larger than the known protein, making the random sequence harder to match.

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

A good minimum length is about 9-10 amino acids, because shorter peptides (6-8) still had perfect matches, while longer ones started to be more unique. Very short peptides are a bad design choice because they are too common and can match normal proteins in the body. This makes it hard for the immune system to tell what is “self” and what is foreign. As a result, it could either attack the body’s own cells or miss real threats. Longer peptides are better because they are more unique and easier to recognize correctly.